



NUOVO MONDO

Pietro Lorenzi, Francesco Lamonaca &
Oumayra Jabeur
2025/26



CATEGORIE DI EQUIPAGGIO INGAGGIABILE

Durante la fase di ingaggio, la struttura dati scelta è un semplice dizionario, dove la chiave è il ruolo, mentre il valore corrisponde al numero di ingaggiati per quel ruolo. Questo dizionario serve solo a essere restituito alla fine della funzione di ingaggio, per poi essere sistemato nella struttura dati vera e propria

Motivo: semplice da assegnare alla vera struttura dati



```
def ingaggia_eq():  
    eq = {  
        "marinaio": 0,  
        "meccanico": 0,  
        "medico": 0,  
        "cuoco": 0,  
        "navigatore": 0  
    }
```

EQUIPAGGIO INGAGGIATO EFFETTIVAMENTE

Struttura dati

Per l'equipaggio abbiamo scelto un dizionario di dizionari, dove alle chiavi, che sono i vari ruoli (marinaio, cuoco ecc...), sono associati dei dizionari che contengono tre campi:

Numero: quante persone di quel ruolo ci sono a bordo

Costo: paga mensile per persona

Morale: Una lista di interi che corrispondono a un valore (il morale) per ogni membro, in modo da gestire facilmente i naufraghi, che hanno un valore di morale diverso dagli altri.

Motivo: Più semplice da gestire negli eventi o nel ciclo

```
equipaggio = {  
  "marinaio": {  
    "numero": 0,  
    "costo": 10,  
    "morale": []  
  },  
  "meccanico": {  
    "numero": 0,  
    "costo": 15,  
    "morale": []  
  },  
  "medico": {  
    "numero": 0,  
    "costo": 25,  
    "morale": []  
  },  
  "navigatore": {  
    "numero": 0,  
    "costo": 20,  
    "morale": []  
  },  
  "cuoco": {  
    "numero": 0,  
    "costo": 15,  
    "morale": []  
  },  
}
```





CATEGORIE DI SCROTE/MERCI ACQUISTABILI

Lo stesso metodo usato con equipaggio l'abbiamo usato poi anche con le scorte e con le merci. Un dizionario che serve solo a mantenere i dati all'interno della funzione, per poi essere restituito alla fine.

Motivo: semplice da assegnare alla vera struttura dati

```
provviste= {  
    "verdura": 0,  
    "frutta": 0,  
    "carne": 0,  
    "acqua": 0,  
}
```

```
merci = {  
    "medicinale": 0,  
    "armi": 0,  
    "sale": 0,  
    "stoffa": 0,  
    "coltelli": 0,  
    "diamanti": 0,  
}
```



MERCI ACQUISTATE EFFETIVAMENTE

Anche qua la struttura dati è sempre un dizionario di dizionari. La chiave è il nome della merce, e nel dizionario associato troviamo

Prezzo: Costo per unità

Numero: La quantità a bordo

Motivo: Più semplice da gestire negli eventi o nel ciclo

```
merci = {  
  "medicinale": {  
    "prezzo": 1,  
    "numero": 0  
  },  
}
```

SCORTE ACQUISTATE EFFETIVAMENTE

Anche per le scorte abbiamo utilizzato un dizionario di dizionari, il quale ha come chiave il nome della provvista e come valore un dizionario che contiene i seguenti campi

Costo: Prezzo per unità

Consumo: Unità consumate per membro settimanalmente

Numero: Quantità a bordo

Il consumo non è fisso ma può variare a seconda delle decisioni dell'utente di raddoppiare o diminuire le dosi settimanali

Motivo: Più semplice da gestire negli eventi o nel ciclo

```
provviste = {  
  "verdura": {  
    "costo": 0.5,  
    "consumo": 0.5,  
    "numero": 0,  
  },  
}
```

STATO DEL GIOCO

Il dizionario "viaggio" raccoglie tutte le variabili globali della partita:

Settimana attuale e Settimane totali: Controllano il ciclo principale

Delta_morale: Il valore positivo o negativo sommato al morale di ogni membro

Scorta dimezzata: flag booleana che serve nel calcolo del rischio di ammutinamento

Conta alabastro: contatore delle volte che l'evento alabastro è stato estratto

Eventi accaduti: lista dei nomi degli eventi già avvenuti

Alabastro avvistato e Alabastro ucciso: flag booleane che influenzano il tradimento e il viaggio di ritorno

Monete: budget rimasto dopo gli acquisti iniziali

Motivazione: Rende più semplice estrarre i dati nel codice ma soprattutto a salvarli su JSON

```
viaggio = {  
  "settimana attuale": 1,  
  "settimane totali": 8,  
  "delta_morale": 0,  
  "scorta dimezzata": False,  
  "conta alabastro": 0,  
  "eventi accaduti": [],  
  "alabastro avvistato": False,  
  "alabastro ucciso": False  
}
```



FILE DI SALVATAGGIO

Il salvataggio avviene su file di testo in formato JSON. Sono disponibili tre slot, corrispondenti ai file saveSlot1.txt, saveSlot2.txt, saveSlot3.txt.

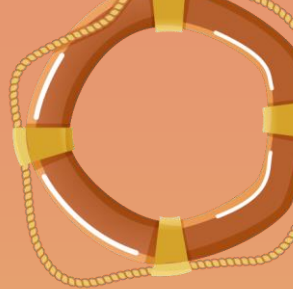
Il file contiene un unico oggetto JSON con quattro chiavi: equipaggio, provviste, merci e viaggio. Ognuna corrisponde al rispettivo dizionario di gioco al momento del salvataggio.

Il caricamento ricostruisce lo stato aggiornando i dizionari con .update() invece di riassegnarli, per garantire che le funzioni definite in core.py continuino a lavorare sugli stessi oggetti in memoria. (tecnica suggerita dall'AI)

```
for ruolo in dati["equipaggio"]:  
    equipaggio[ruolo].update(dati["equipaggio"][ruolo])
```

DATI CHE VENGONO SALVATI:

- Equipaggio
- Provviste
- Merci
- Viaggio
- Eventi accaduti



GESTIONE EVENTI NON RIPETIBILI

Il dizionario viaggio contiene una lista chiamata eventi accaduti. Ogni volta che un evento viene eseguito, il suo nome (ricavato tramite evento.__name__) viene aggiunto alla lista (idea nostra, soluzione dall'AI).

Prima di eseguire un evento, il ciclo principale verifica che il suo nome non sia già presente in eventi accaduti. Se lo è, viene estratto un altro evento dalla pool. In questo modo ogni evento può accadere al massimo una volta.

L'evento avvistamento_alabastro è trattato separatamente: invece di essere aggiunto alla lista, viene usato un contatore conta alabastro che permette fino a 3 ripetizioni. L'evento nessun_imprevisto non viene mai aggiunto alla lista e può accadere un numero illimitato di volte.

USO DELL'AI



L'AI più utilizzata è stata Claude, specificamente nei seguenti ambiti:

- Revisione e debugging del codice
- Gestione e aggiornamento del morale
- Storytelling del gioco
- Creazione del file .exe

L'AI è stata utilizzata come supporto, le decisioni delle strutture dati e degli algoritmi è stata quasi interamente in mano a noi.

